

Homework 1

Moida Praneeth Jain

2022101093

August 7, 2025

1) Scalable Eventually Consistent Counters over Unreliable Networks

1 (a) What is the system/theory/result about?

The paper introduces a way to implement eventually consistent counters. It addresses the need for such scalable, fault-tolerant counters for large-scale distributed applications operating over unreliable networks. It first defines a relaxed eventual consistency model, called an Eventually Consistent Distributed Counter (ECDC). The primary result is the design of a new Conflict-free Replicated Data Type (CRDT) called the *Handoff Counter*, which implements the ECDC model while avoiding the scalability issues that come with naive CRDT approaches.

1 (b) What is the main result?

The main result is the *Handoff Counter*. This mechanism solves the “ID explosion problem”: where the state of a naive CRDT counter grows indefinitely with the number of participating nodes. This is done by utilizing the following two techniques:

- **Hierarchical Tiers:** Nodes are organized into layers, where ephemeral clients (e.g., mobile devices) hand off their state to a fixed set of permanent server nodes. This ensures the system’s state remains bounded.
- **Reliable Handoff Mechanism:** A 4-way handshake protocol within the CRDT’s state ensures that counted values are transferred from clients to servers without loss or duplication (exactly-once semantics), even across unreliable networks.

The paper ends by formally proving that the Handoff Counter correctly implements the ECDC specification.

1 (c) What is the significance, what problem does it solve?

This is significant because it provides a practical and theoretically sound solution to the problem of distributed counting at scale. It directly solves the conflict between the need for high availability and the need for scalability. Prior solutions were either unavailable during network partitions (strong consistency models) or unscalable due to state growth (naive CRDTs).

The Handoff Counter resolves this by providing a mechanism that is simultaneously available, partition-tolerant, and scalable.

1 (d) How do you find it useful?

The system is useful as a reusable architectural pattern for managing distributed state generated by clients.

- **System Design:** The hierarchical model is a general strategy for defending the core of a distributed system against unbounded state growth caused by a large number of clients.
- **Application Development:** The Handoff Counter provides a straightforward method for implementing features like voting, likes, etc in large-scale applications.
- **Theoretical Framework:** The ECDC criteria offer a clear, formal definition of correctness for weakly consistent counters. Such a criteria might come in handy for future research.

1 (e) What do you find interesting in the system/result?

The most interesting aspect of the result in my opinion is the way in which the communication protocol is embedded within the data structure itself.

- **Embedded Protocol:** The reliability of the state transfer is not handled by an external protocol but is an inherent part of the CRDT's state (slots, tokens) and merge logic..
- **Solution to State Bloat:** The handoff mechanism solves a well-known limitation in CRDTs. By allowing temporary state to be garbage collected after being securely merged into the permanent core, it makes the CRDT approach viable for a much wider range of applications.

1 (f) How can it be improved in some direction?

The proposed system could be extended in a few directions:

- **Resilience to Permanent Client Failure:** The current design does not fully address the issue of a client failing permanently after initiating a handoff, which would leave an orphaned slot on the server. A possibility would be to incorporate timeout mechanism, allowing servers to garbage-collect stale slots after a certain period of inactivity.
- **Generalization:** The handoff mechanism can be generalized to other commutative data types. An extension would be to formally define and prove the correctness of these generalized structures.
- **Implementation Complexity:** The merge logic is composed of eight transformations, which may be complex to implement correctly. The existence of simpler merge models providing the same guarantees could be explored.

2) Analysis of TCP and UDP Protocols

2 (a) Asynchronous vs. Synchronous Communication

TCP is a **synchronous** protocol, whereas UDP is **asynchronous**.

- **TCP (Synchronous):** It has a connection-oriented model. Before transmission, a three-way handshake is mandatory. The entire data transfer happens within the context of this synchronized session.
- **UDP (Asynchronous):** It has a connectionless fire-and-forget model. A process can send a UDP datagram at any time without any handshake with the receiver.

NOTE: For a single `send()` primitive, both protocols behave asynchronously, as the call returns to the application without waiting for the remote host to `recv()` it.

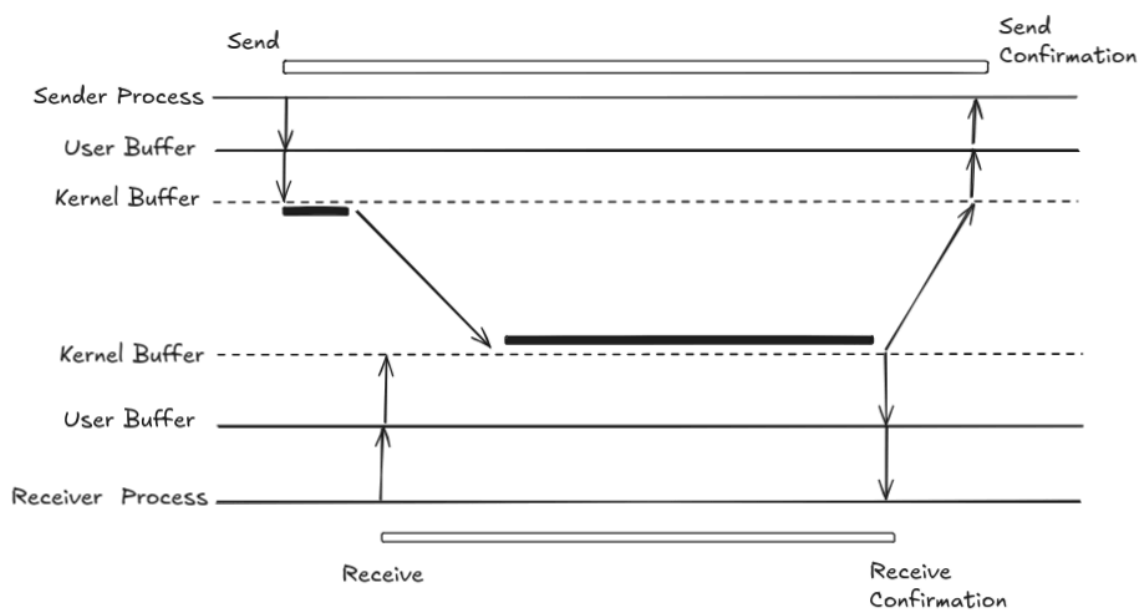
2 (b) Blocking vs. Non-blocking Communication

This is a configurable behavior of the socket API, not the protocols themselves. An application can configure a socket for either TCP or UDP to operate in either of the two modes:

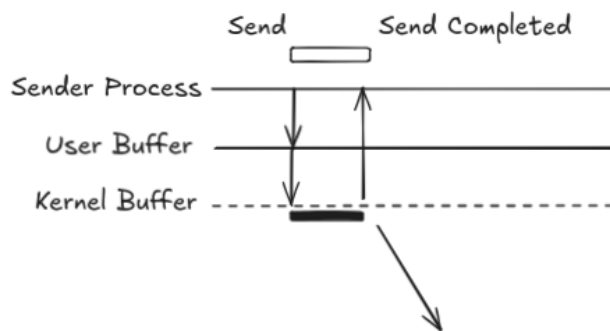
- **Blocking (Default):** A blocking call will suspend the execution of the application process until the operation is complete. For a `send()`, this means waiting until the data has been copied from the user buffer to the kernel buffer. For a `recv()`, this means waiting until the data arrived in the kernel buffer has been copied into the user buffer.
- **Non-blocking:** A non-blocking API call returns control to the process immediately, regardless of whether the operation could be completed.

2 (c) Communication Model Diagrams

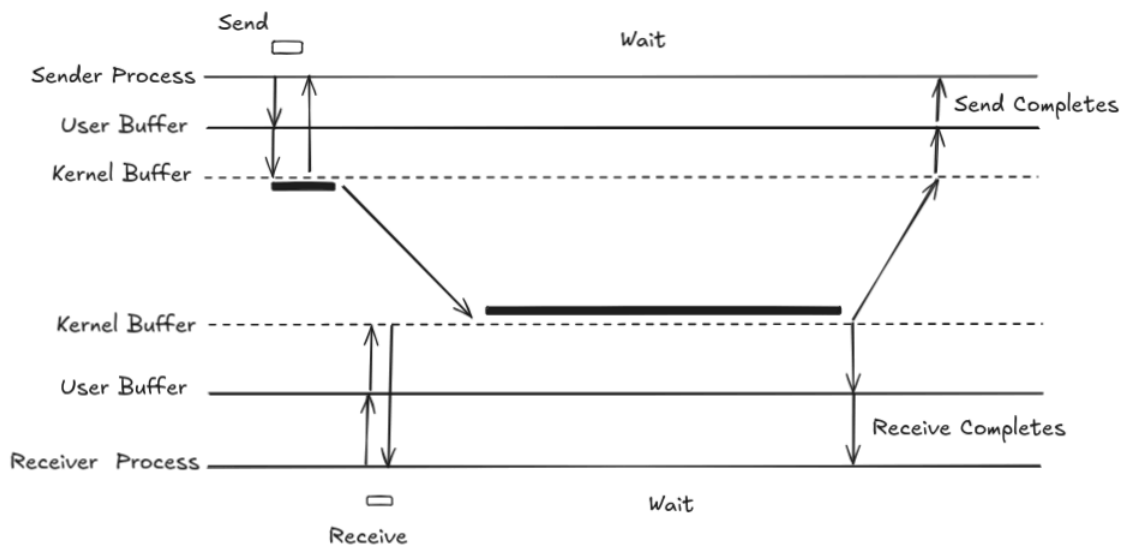
TCP (synchronous) with blocking send and receive:



UDP (asynchronous) with blocking send:



TCP (synchronous) with non-blocking send and receive



UDP (asynchronous) with non-blocking send:

